

ProcureAI – The Autonomous Procurement War Room

Business Proposal · SMYA Hackathon (AWS AI Agents) · Team code PCNEI7FK · Problem statement: Supplier Comparison

Team code	PCNEI7FK
Live deployment	http://47.129.120.76/ (AWS Lightsail, ap-southeast-1)
Repository	https://github.com/gaamiranda/hackathon
Demo video	https://www.youtube.com/watch?v=QoNaI5Gtsxg

1. The problem

Problem statement (Supplier Comparison), as issued: "Procurement executives regularly receive quotations from multiple suppliers for the same products. Before every purchase, they manually compare prices, delivery lead times, payment terms, and supplier performance using spreadsheets and emails. As the number of suppliers and products grows, the comparison process becomes increasingly difficult, resulting in slower purchasing decisions and missed opportunities to negotiate better terms."

A buyer asks a simple question: *which supplier should we use for this order?* Answering it today means opening three quotations that all look different (a PDF, a spreadsheet, an email), copying prices, minimum order quantities, lead times and payment terms into a comparison sheet, looking up how each supplier performed last time, and deciding. It takes 15 to 30 minutes per comparison, it is done under time pressure, and small orders are almost never negotiated because nobody has the time. Errors are invisible: a wrong total printed on a quotation is copied as-is, and the cheapest-looking supplier wins even when its delivery record says it should not.

The obvious answer, "let an AI compare the quotes", fails the moment a finance director asks how the number on the screen was produced. A language model that adds up totals will eventually get one wrong, negotiate in circles, tell one supplier what another one offered, or follow an instruction hidden inside a supplier's email. Enterprise procurement cannot accept any of those.

2. The proposal

ProcureAI is an **AI procurement team, not a chatbot**. Four specialised agents read the quotations, look up supplier history, explain a recommendation, and negotiate with suppliers, while a deterministic engine owns every number and a human approves every consequential action. The buyer supervises a team; they no longer do the work.

One rule runs through the design: the agent runtime (OpenClaw on AWS) runs the agents, Python calculates the truth, and the language model (Claude Sonnet 4.5 on Amazon Bedrock) only extracts, interprets, explains and drafts. It never performs arithmetic that matters, never scores, and never acts without a gate.

What happens in a run

1. The buyer states the need: 2,000 units of Product X in 14 days, budget 30,000 USD.

2. Three supplier quotations are dropped in. The Document Agent extracts each into structured data with a confidence per field, and an independent judge model verifies each critical value against the document.
3. The engine recomputes every total. One quotation prints 22,040; the arithmetic says 22,800. The workflow stops and asks the buyer.
4. Supplier history (on-time rate, defect rate, capacity, blacklist) comes from an authoritative store. The engine scores every eligible quote; the Decision Agent explains the ranking in prose that may not contain a number it was not given.
5. The Negotiation Agent drafts price and lead-time requests for the top suppliers: two rounds maximum, nothing sent without the buyer's approval, and every draft filtered so no supplier ever learns another's terms. Counter-offers are re-scored.
6. Mid-run, the order changes to 5,000 units. The team replans on the quotes it already has: capacity, minimum quantities, prices, lead times, budget and risk are revisited, the recommendation flips, and the change is explained.
7. The buyer approves the final supplier and only then is the purchase order generated.

3. Who it is for and what it is worth

Stakeholder	Today	With ProcureAI
Buyer / procurement specialist	15–30 min per comparison; no negotiation on small orders	Under a minute to a ranked, explained recommendation; every order negotiated within policy; the buyer only approves
Procurement manager	Decisions in spreadsheets, rationale in someone's head	Every run is an auditable timeline: what each agent saw, what the engine computed, what the human approved
Finance / audit	Cannot verify how a supplier was chosen or a total produced	Every number traces to a formula and an extracted input; model prose is guarded against invented figures
Security / risk	AI tools that act on whatever text they read	Supplier content is untrusted data end to end; injections are detected and ignored; the runtime agent has no tools

4. Success measures (and what we measured)

We deliberately did not define success as "the AI is always right". We defined it as: fast where speed is safe, blocked where it is not, and human-approved where it matters.

Metric	Baseline	Target	Result in the built system
Quote processing time (upload to recommendation)	15–30 min manual	< 30 s	Three documents through Claude in 12–15 s sequentially; under 2 s when replayed from cache
Critical-field safety	Errors copied silently	100% of low-confidence or unsupported critical fields blocked from autonomous progression	Achieved: a low-confidence or judge-rejected field always routes to the human extraction form; a tampered unit price scored 0.01 supported by the judge and was blocked

Arithmetic correctness	Trust the document	Every total recomputed; mismatch stops the workflow	Achieved on every run (22,040 stated vs 22,800 computed → stop). The prose guard caught the model computing figures that were not in its input on four separate occasions during development; each time the deterministic text stood
Negotiation policy compliance	n/a	100%	Third round unreachable by design; competitor names or prices in an outgoing message are blocked (regex and judge, 0.96 leaking vs 0.11–0.26 clean); all four recorded drafts were clean
Consequential actions human-approved	n/a	100%	Sending a negotiation message and generating a PO require a click; no agent has a tool that can do either
Replanning success	Start over	New valid recommendation without restart	Quantity 2,000 → 5,000 replanned in under 1 s with a per-supplier impact table; a second scenario with a budget cut behaves the same way

5. Trust and controls

We started from a pre-mortem, "why would this fail in front of a CFO?", and built one control per failure.

Failure mode	Control	Evidence
Math hallucination	Deterministic Python computes every total, score and check; model totals are only compared	Mismatch gate fires on every run with the faulty quotation
Invented figures in explanations	A number guard rejects any model prose containing a number absent from its input; deterministic text is used instead, visibly	Caught real cases during development (a "saving" and a "50% weight" that did not exist)
Infinite negotiation	Two rounds per supplier, enforced by the workflow state machine, not by the prompt	Tested: a forced third round is refused
Confidential data leakage	Outbound filter plus an independent judge before the human gate; human edits are re-checked	Edited draft naming a competitor rejected with the reason shown
Prompt injection in supplier content	Documents and replies are delimited as data; the agent has no tools; the judge flags injections	Injection text scored 0.94–0.96 vs 0.03–0.06 for clean text; extraction unchanged on every test
AI unavailable	Automatic fallback route, then manual mode with the same engine numbers	Seven failure drills executed and documented, including killing the runtime and the model mid-demo

6. What was built and deployed

- **Live system** on AWS Lightsail with OpenClaw as the agent runtime, following the sponsor starter kit; every agent call routed through a dedicated, tool-less OpenClaw agent, with an automatic fallback to

the AWS LLM gateway. Runs persist across restarts.

- **The War Room:** a live view of the agent team (one lane per agent), the timestamped timeline of every action, highlighted "moments" (math stopped, injection caught, message blocked, requirement changed, recommendation changed, PO generated), and the human gates.
- **Two independent scenarios:** industrial widgets with a quantity change, and stainless fasteners with a blacklisted supplier and a budget cut, on the same code.
- **313 automated tests** that run offline against recorded live model responses, so what they guarantee is the real model's behaviour.

7. Roadmap

Horizon	Step
Next 4 weeks	Real inbound channels (mailbox, supplier portal) feeding the same untrusted-text pipeline; supplier history in DynamoDB; https and SSO on the deployment
Next quarter	Multi-line and multi-product requests; ERP purchase-order export; negotiation boundaries learned from historical outcomes, still enforced deterministically
Commercial	Per-seat subscription for mid-size buying teams that handle 50–500 comparisons a month, where a 15-minute saving per comparison and negotiated savings on orders that were never negotiated pay for the tool within the first month

8. Team

Team PCNEI7FK. Built in three weeks using a written plan of record (PLAN.md in the repository) that lists every design decision, and a task-by-task delegation to coding agents with human review of every change.